%.o: %.c
$(NAME):
.PHONY

42 MULHOUSE

Makefile

Compiler sans se répéter, ni tout recompiler

eschwart

POURQUOI MAKE

Le problème

Un petit projet : **main.c**, **parser.c** et **utils.c**. Pour compiler, il faut taper :

```
$ cc -Wall -Wextra -Werror main.c parser.c utils.c -o program
```

01

Une commande longue

À retaper à chaque compilation.

02

Facile de se tromper

Un fichier ou un flag oublié, et le résultat change.

03

Tout est recompilé

Même les fichiers qui n'ont pas bougé depuis hier.

LES RÈGLES

Votre premier Makefile

Makefile

```
program:
    cc main.c utils.c parser.c -o program
```

Terminal

```
$ make
cc main.c utils.c parser.c -o program
```

make lance la première règle

Taper make tout court exécute la première cible du fichier.

La recette commence par une tabulation

Pas des espaces. Sinon : **missing separator. Stop.**

Une limite

Modifie main.c et relance make : « 'program' is up to date ». Sans dépendances, make ne sait pas que main.c a changé.

LES RÈGLES

Les dépendances

La forme d'une règle

```
target: prerequisites
    recipe
```

Makefile

```
program: main.o utils.o parser.o
    cc main.o utils.o parser.o -o program

main.o: main.c
    cc -c main.c -o main.o

utils.o: utils.c
    cc -c utils.c -o utils.o

parser.o: parser.c
    cc -c parser.c -o parser.o
```

L'arbre des dépendances

```
program
├── main.o
│   └── main.c
├── utils.o
│   └── utils.c
└── parser.o
    └── parser.c
```

- une cible dépend d'autres fichiers
- si une dépendance est plus récente que la cible, la recette est relancée
- make remonte l'arbre avant d'exécuter quoi que ce soit
- les .h aussi sont des dépendances (on automatise ça plus loin)

LES RÈGLES

Le relink

Makefile

```
program:
    gcc main.c utils.c parser.c -o program_name
```

Terminal

```
$ make
gcc main.c utils.c parser.c -o program_name
$ make
gcc main.c utils.c parser.c -o program_name
```

Le 2e **make** recompile tout alors que rien n'a changé : c'est un **relink**.

Make compare la date de la cible avec celle de ses dépendances. Ici, aucun fichier « program » n'existe : la règle est relancée à chaque fois.

Les causes classiques

- la cible ne porte pas le nom du fichier produit
- $(NAME) dépend d'une cible .PHONY (par exemple « libft »)
- une règle qui recrée un fichier à chaque appel

Le test : lancer **make** deux fois. La 2e fois doit afficher Nothing to be done for 'all'.

LES RÈGLES

Les règles génériques

Avant : une règle par fichier

```
main.o: main.c
    cc -c main.c -o main.o
utils.o: utils.c
    cc -c utils.c -o utils.o
parser.o: parser.c
    cc -c parser.c -o parser.o
```

devient

Après : une seule règle générique

```
%.o: %.c
    cc -c $< -o $@
```

Symbole	Signification	Pour main.o
%	la partie variable du nom	main
$@	la cible	main.o
$	la première dépendance	main.c
$^	toutes les dépendances	main.c

Comment make la lit

Pour construire un .o, il faut le .c du même nom. Pour main.o, la règle devient :

```
main.o: main.c
    cc -c main.c -o main.o
```

LES RÈGLES

Le lien (linking)

Makefile

```
program: main.o utils.o parser.o
    cc $^ -o $@
```

Avec ces dépendances, la commande exécutée devient :

Ce que make lance

```
cc main.o utils.o parser.o -o program
```

Le principe

- l'exécutable dépend de tous les .o
- si un seul .o change, make relance le lien
- le lien assemble les .o en un programme

$^ et $@

$^ = toutes les dépendances, $@ = la cible. Ajouter un fichier ne demande plus de toucher à la recette.

LES VARIABLES

Les variables

Makefile

```
CC     = cc
CFLAGS = -Wall -Wextra -Werror

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

program: main.o utils.o parser.o
    $(CC) $^ -o $@
```

Ce que make lance

```
cc -Wall -Wextra -Werror -c main.c -o main.o
cc -Wall -Wextra -Werror -c utils.c -o utils.o
cc -Wall -Wextra -Werror -c parser.c -o parser.o
```

Une valeur écrite une seule fois, utilisée partout : le Makefile est plus lisible et se modifie à un seul endroit.

Opérateur	Comportement	Exemple
=	évaluée à chaque utilisation	CC = cc
:=	évaluée une fois, tout de suite	D := $(shell date)
?=	seulement si pas déjà définie	CC ?= cc
+=	ajoute à la suite	CFLAGS += -g

On peut aussi surcharger une variable au lancement : **make CC=clang**.

LES VARIABLES

Générer la liste des .o

Makefile

```
CC      = cc
CFLAGS = -Wall -Wextra -Werror

SRC     = main.c utils.c parser.c
OBJ     = $(SRC:.c=.o)

program: $(OBJ)
    $(CC) $(OBJ) -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

SRC → OBJ

SRC liste les .c. **$(SRC:.c=.o)** remplace chaque .c par .o :

`OBJ = main.o utils.o parser.o`

La règle implicite

make connaît déjà %.o: %.c (avec $(CC) et $(CFLAGS)). Écris-la quand même : on voit ce qui se passe, et elle devient indispensable dès que les .o vont dans un dossier obj/.

Liste tes fichiers à la main

Beaucoup d'évaluateurs refusent $(wildcard *.c).

LES VARIABLES

Les directives spéciales

.PHONY

```
.PHONY: clean

clean:
    rm -f $(OBJ)
```

Par défaut, une cible est un fichier. Si un fichier nommé clean existe, make le croit à jour et ne fait rien. .PHONY dit : ce n'est pas un fichier.

@

```
clean:
    @rm -f $(OBJ)
```

Devant une commande : make ne l'affiche pas, mais sa sortie et ses erreurs restent visibles.

.SILENT

```
.SILENT:
```

Masque toutes les commandes. Joli, mais en développement et en évaluation on veut voir ce qui est lancé : à éviter pour débuter.

UN MAKEFILE COMPLET

Un Makefile complet, conforme 42

Variables et compilation

```
NAME    = program
CC      = cc
CFLAGS  = -Wall -Wextra -Werror

SRC     = main.c utils.c parser.c
OBJ     = $(SRC:.c=.o)
DEP     = $(OBJ:.o=.d)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $(NAME)

%.o: %.c
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@
```

Règles obligatoires

```
clean:
    rm -f $(OBJ) $(DEP)

fclean: clean
    rm -f $(NAME)

re: fclean
    $(MAKE) all

-include $(DEP)

.PHONY: all clean fclean re
```

- **all** en première règle : c'est la cible par défaut
- **-MMD -MP** : recompile quand un .h change
- **re** rappelle $(MAKE) : sûr avec make -j
- **cc** : le compilateur demandé par les sujets

ORGANISER SON PROJET

Découper un Makefile

Makefile

```
include mk/variables.mk
include mk/sources.mk
include mk/rules.mk
```

Arborescence

```
project/
├── Makefile
├── mk/
│   ├── variables.mk
│   ├── sources.mk
│   └── rules.mk
├── src/
└── include/
```

mk/sources.mk

```
SRC = src/main.c \
      src/parser.c \
      src/utils.c

OBJ = $(SRC:.c=.o)
```

mk/rules.mk

```
$(NAME): $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $(NAME)

%.o: %.c
    $(CC) $(CFLAGS) $(INC) -c $< -o $@
```

Un fichier par responsabilité : le Makefile principal reste lisible et chaque partie évolue seule.

ORGANISER SON PROJET

Compiler avec sa libft

Makefile

```
LIBFT   = libft/libft.a

$(NAME): $(OBJ) $(LIBFT)
    $(CC) $(CFLAGS) $(OBJ) $(LIBFT) -o $(NAME)

$(LIBFT):
    $(MAKE) -C libft

clean:
    $(MAKE) -C libft clean
    rm -f $(OBJ) $(DEP)

fclean: clean
    $(MAKE) -C libft fclean
    rm -f $(NAME)
```

make -C libft

make se place dans le dossier libft et y lance son propre Makefile.

Dépendre du fichier libft.a

Pas d'une cible .PHONY « libft » : elle serait toujours à refaire, donc relink à chaque make.

Nettoyer aussi la libft

clean et fclean appellent ceux de la libft.

AUTOMATISER

Make comme task runner

Makefile

```
test: $(NAME)
    ./tests/run_tests.sh

valgrind: $(NAME)
    valgrind --leak-check=full ./$(NAME)

norm:
    norminette src include

run: $(NAME)
    ./$(NAME)

.PHONY: test valgrind norm run
```

Make ne sert pas qu'à compiler : il peut lancer toutes les tâches répétitives du projet. On parle de **task runner**.

Pourquoi c'est pratique

- une seule commande à retenir : make test
- le même comportement pour toute l'équipe
- dépendre de $(NAME) recompile avant de tester

Ces cibles ne sont pas des fichiers : pense au **.PHONY**.

AUTOMATISER

Automatiser Git

Makefile

```
status:
    git status

push:
    git add -A
    git commit -m "$(MSG)"
    git push
```

Terminal

```
$ make push MSG="fix parsing"
```

Le .gitignore d'abord

Sans lui, git add -A envoie aussi les .o, les .d et le binaire.

.gitignore

```
*.o
*.d
program
```

Et sans MSG ?

Le message est vide : git refuse le commit (« Aborting commit due to empty commit message »).

AUTOMATISER

Git interactif : les pièges

Commit interactif

```
push:
    @git status --short
    @printf 'Commit et push ? [y/N] '; read ans; \
    if [ "$$ans" = "y" ]; then \
        printf 'Message : '; read msg; \
        git add -A && git commit -m "$$msg" && git push; \
    else \
        echo "Annulé"; \
    fi
```

make branch-parsing

```
branch-%:
    git switch -c $*
    git push -u origin $*
```

Une ligne = un shell

Chaque ligne de recette tourne dans son propre shell : pour garder une variable, on enchaîne avec ; et \.

$$ pour le shell

make remplace $; $$ce laisse passer $ce au shell.

$* dans branch-%

Vaut ce que % a capturé : parsing.

Attention

Une cible qui fait git reset --hard ou git branch -D efface du travail sans retour : toujours demander confirmation.

POUR ALLER PLUS LOIN

Petits bonus

PETITS BONUS

Organiser les sources avec addprefix

Exemple : webserv

```
# Fichiers par module
CGI_FILES    = Cgi.cpp
CONFIG_FILES = Config.cpp Location.cpp ServerConfig.cpp
HTTP_FILES   = HttpRequest.cpp HttpResponse.cpp
SERVER_FILES = Client.cpp Server.cpp Router.cpp

# Ajouter le dossier devant chaque fichier
CGI    = $(addprefix srcs/cgi/, $(CGI_FILES))
CONFIG = $(addprefix srcs/config/, $(CONFIG_FILES))
HTTP   = $(addprefix srcs/http/, $(HTTP_FILES))
SERVER = $(addprefix srcs/server/, $(SERVER_FILES))

SRC    = $(CGI) $(CONFIG) $(HTTP) $(SERVER)
```

Ranger les fichiers par module et laisser make construire les chemins complets.

Ce que ça donne

```
HTTP = srcs/http/HttpRequest.cpp
srcs/http/HttpResponse.cpp
```

Ajouter un fichier : une seule ligne à toucher.

PETITS BONUS

Compiler en parallèle

Makefile

```
MAKEFLAGS += -j$(shell nproc)
```

Le principe

make lance autant de compilations en même temps que la machine a de cœurs. Sur un gros projet, le gain est énorme.

La condition

Des dépendances exactes : deux règles qui dépendent l'une de l'autre sans le dire peuvent se lancer en même temps.

Sur Mac

nproc n'existe pas : sysctl -n hw.ncpu donne le nombre de cœurs.

PETITS BONUS

re et la compilation parallèle

⚠ À éviter

```
re: fclean $(NAME)
```

Avec **make -j**, make lance fclean et $(NAME) **en même temps** : des .o sont supprimés pendant qu'on les compile. Le build casse, ou ne casse qu'une fois sur deux.

✓ Version sûre

```
re: fclean
	$(MAKE) all
```

fclean se termine d'abord, puis un **nouveau make** repart de zéro. Aucune course possible, même en parallèle.

Dans une recette, écris toujours **$(MAKE)** et jamais make : il transmet les options (-j, -s…) au make appelé.

PETITS BONUS

Trois finitions

.DEFAULT_GOAL

```
.DEFAULT_GOAL := all
```

Fixe la cible lancée par make tout court, quel que soit l'ordre des règles.

Des messages

```
$(NAME): $(OBJ)
    @printf "Linking...\n"
    $(CC) $(OBJ) -o $@
```

Une sortie plus lisible pour suivre les étapes du build.

-MMD -MP

obj/main.d, généré par cc

```
obj/main.o: src/main.c include/prog.h
include/prog.h:
```

cc écrit les headers de chaque .c dans un .d, que -include relit. -MP évite une erreur si un .h est supprimé.

Kahoot !

Sortez vos téléphones

Merci !

Des questions ?

eschwart · 42 Mulhouse